

CSCI2750 Data Mining & Information Processing

Lecture 11: Frequent Pattern Mining

Wai-Yiu Keung

Term 2, 2025-26

Department of Computer Science and Engineering
The Chinese University of Hong Kong

Motivation: Market Basket

TID	Itemset
1	$\{bread, milk\}$
2	$\{bread, diapers, beer, eggs\}$
3	$\{milk, diapers, beer, coke\}$
4	$\{bread, milk, diapers, beer\}$
5	$\{bread, milk, diapers, coke\}$

- consider a transaction dataset in a certain grocery store
- here m rows means m transactions, and each itemset means what are bought in that transaction
- question: what is the rules between purchases of different products?
 - e.g., $\{diapers\}$ seems to be associated with $\{beer\}$
 - also, it seems that $\{bread\}$ or $\{beer\}$ are linked with $\{milk\}$
- this is called the **market basket problem**

Motivation: Singer Recommendation

User	Singers in Playlist
Chris	Serrini, Terence
Eve	Serrini, Billy, Terence, Jace
Frankie	Terence, Billy, Jace
Grace	Terence, Jace
Harry	Serrini, Terence, Jace
Kent	Terence, Billy, Jace
Mary	Serrini, Jace

- consider a dataset of the singers in the playlist of several people, who possible are from the same group; e.g., you know they are all from the same major
- question: what are the singer preference pattern?
 - e.g., Serrini fans appear to also be a fan of Terence
 - if another student from that major has Terence and Billy in his/her playlist, we probably want to recommend the singer Jace to that student

Motivation: Library Information System

User ID	Borrowing Record
1	$\{a, d\}$
2	$\{b, c, d\}$
3	$\{a, b, c, d\}$
4	$\{a, c, d\}$
5	$\{b, c, d\}$
6	$\{b, d\}$
7	$\{c, d\}$

- consider a small library record with merely 4 books (encoded as a, b, c, d resp.)
- if you borrowed book b , it is almost certain that you will someday borrow book d too
- above we've seen many applications of the same task: frequent pattern mining

Basic Set-up

$\mathcal{T} =$	TID	Itemset
	1	$\{bread, milk\}$
	2	$\{bread, diapers, beer, eggs\}$
	3	$\{milk, diapers, beer, coke\}$
	4	$\{bread, milk, diapers, beer\}$
	5	$\{bread, milk, diapers, coke\}$

- $\mathcal{T} = \{T_1, \dots, T_N\}$ is the collection of transactions recorded in the dataset
- $\mathcal{U} = \{u_1, u_2, \dots, u_d\}$ is a collection of d **unique items** appeared in a transaction dataset
- As an example, we describe

$$\mathcal{U} = \{bread, milk, diapers, beer, coke, eggs\}$$

and the transactions are $\mathcal{T} = \{T_1, \dots, T_5\}$ with T_n being the n -th row of it.

- A collection of i 's constitutes to an **itemset** $\mathcal{I}$, with k -itemset denoting a collection of k items from the set $\mathcal{I}$. For example, $\{milk, diapers, beer\}$ is a 3-itemset.

Basic Set-up

We are interested in measuring “how frequent” does an itemset $\mathcal{I}$ appear in $\mathcal{T}$. We say
support count of $\mathcal{I} = \sigma(\mathcal{I}) =$ number of occurrence of an $\mathcal{I}$ in $\mathcal{T}$.

Also, define the measurement:

$$\text{support} = \text{sup}(\mathcal{I}) = \frac{\sigma(\mathcal{I})}{|\mathcal{T}|}.$$

Note that this is the *frequency* of $\mathcal{I}$ in $\mathcal{T}$. To illustrate, consider again the example dataset. We have

$$\begin{aligned}\sigma(\{milk, diapers, beer\}) &= 2, \\ \text{sup}(\{milk, diapers, beer\}) &= 2/5 = 40\%.\end{aligned}$$

Given a predefined threshold $\text{minsup} \in (0, 1)$, an itemset $\mathcal{I}$ is said to be **frequent** if

$$\text{sup}(\mathcal{I}) \geq \text{minsup}.$$

Association Rule

Define an **associate rule** r as

$$r : \mathcal{I}_1 \rightarrow \mathcal{I}_2$$

where both $\mathcal{I}_1, \mathcal{I}_2 \subset \mathcal{U}$; and that $\mathcal{I}_1 \cap \mathcal{I}_2 = \emptyset$.

Examples:

$$\begin{aligned} \{diapers\} &\rightarrow \{beer\} \\ \{milk, bread\} &\rightarrow \{eggs, coke\} \\ \{beer, bread\} &\rightarrow \{milk\} \end{aligned}$$

are some example association rules in $\mathcal{T}$.

On the other hand,

$$\begin{aligned} \{beer, lemon\} &\rightarrow \{diapers\} \\ \{beer, diapers\} &\rightarrow \{diapers\} \end{aligned}$$

are **not** association rules. This is so because i) Lemon does not exist in $\mathcal{U}$ and ii) Diapers exists in both sides.

Support and Confidence

Given a rule $r : \mathcal{I}_1 \rightarrow \mathcal{I}_2$, we need to measure “how strong” the rule is.

Define the **support of r** as

$$\text{sup}(r) = \text{sup}(\mathcal{I}_1 \cup \mathcal{I}_2).$$

Some intuitions:

- a small $\text{sup}(r)$ means only a few transactions obey r ; in such case r is rather uninteresting
- a large $\text{sup}(r)$ means the *coexistence* of $\mathcal{I}_1$ and $\mathcal{I}_2$ is **popular** in $\mathcal{T}$, and hence such r is of higher interest.

Also, define the **confidence of r** as

$$\text{conf}(r) = \frac{\text{sup}(r)}{\text{sup}(\mathcal{I}_1)} = \frac{\text{sup}(\mathcal{I}_1 \cup \mathcal{I}_2)}{\text{sup}(\mathcal{I}_1)}.$$

- $\text{conf}(r)$ reflects the reliability of the inference made by r
- interpret the formula as:

$$\text{conf}(r) = \frac{\text{sup}(\mathcal{I}_1 \cup \mathcal{I}_2)}{\text{sup}(\mathcal{I}_1)} = \frac{\text{Pr}(\text{both } \mathcal{I}_1 \text{ and } \mathcal{I}_2 \text{ occur in a transaction})}{\text{Pr}(\text{only } \mathcal{I}_1 \text{ exists in a transaction})}.$$

That is, $\text{conf}(r) = 1$ if every transactions containing $\mathcal{I}_1$ also contains $\mathcal{I}_2$. Similarly, $\text{conf}(r) = 0$ if every transactions containing $\mathcal{I}_1$ does not have $\mathcal{I}_2$.

Example

TID	Itemset
1	$\{bread, milk\}$
2	$\{bread, diapers, beer, eggs\}$
3	$\{milk, diapers, beer, coke\}$
4	$\{bread, milk, diapers, beer\}$
5	$\{bread, milk, diapers, coke\}$

Let $\mathcal{I}_1 = \{milk, diapers\}$ and $\mathcal{I}_2 = \{beer\}$. Consider a rule

$$r : \mathcal{I}_1 \rightarrow \mathcal{I}_2 = \{milk, diapers\} \rightarrow \{beer\}.$$

It has the following support and confidence measures:

$$\begin{aligned}\text{sup}(r) &= \frac{\sigma(milk, diapers, beer)}{|\mathcal{T}|} = \frac{2}{5} \\ \text{conf}(r) &= \frac{\text{sup}(r)}{\text{sup}(milk, diapers)} = \frac{2/5}{3/5} = \frac{2}{3}\end{aligned}$$

Therefore, 40 percents of the transactions consists of both items inside $\mathcal{I}_1$ and $\mathcal{I}_2$; and among those with $\mathcal{I}_1$, about 67 percents of them will purchase items in $\mathcal{I}_2$, too.

Association Rule Mining

Finally, we are ready to formally define of the task of association rule mining:

Definition. (Association Rule Mining).

Given a set of transactions $\mathcal{T}$, find all the rules r that satisfies

- $\text{sup}(r) \geq \text{minsup}$; and
- $\text{conf}(r) \geq \text{minconf}$

where $\text{minsup} \in (0, 1)$ and $\text{minconf} \in (0, 1)$ are some predefined minimum threshold values of support and confidence.

We attempt to separate our task into two stages.

- First, find all potential itemset to serve as $\mathcal{I}_1 \cup \mathcal{I}_2$, so that $\text{sup}(r)$ is always no less than minsup ; This step is called **frequent itemset mining**.
- Second, for each frequent itemset returned from the above step, we perform **rule generation** by checking whether the confidence of each candidate rule satisfies minconf .

We will first focus on the freq. itemset mining.

Association Rule Mining: A Brute-Force Approach

Basic Idea:

- given $\mathcal{U}$, list all possible candidate itemsets out (about $2^{|\mathcal{U}|}$ candidates $\mathcal{I}$)
- compute all the corresponding $\text{sup}(\mathcal{I})$; then, consider only $\mathcal{I}$ with $\text{sup}(\mathcal{I}) \geq \text{minsup}$
- generate possible rules from each frequent $\mathcal{I}$, and calculate the corresponding $\text{conf}(r)$
- eliminate all rules that does not satisfy $\text{conf}(r) \geq \text{minconf}$

This approach is referred to as the **Brute-force** approach, which is also known as **greedy search**.

Imaginably, the greater number of transactions in $\mathcal{T}$ and elements of unique items $\mathcal{U}$ leads to an expansion of the computational time required for completing the search.

Brute-Force: Example

Consider a simple dataset:

$$\mathcal{T} = \begin{array}{c|c} \text{TID} & \text{itemset} \\ \hline 1 & \{a, b\} \\ 2 & \{a\} \\ 3 & \{a, b, c\} \\ 4 & \{b\} \\ 5 & \{b, c\} \end{array} \implies \mathcal{U} = \{a, b, c\}$$

The corresponding candidate itemsets & their supports can be tabulated as:

a	b	c	$\mathcal{I}$	$\text{sup}(\mathcal{I})$
0	0	0	$\emptyset$	0
0	0	1	$\{c\}$	2/5
0	1	0	$\{b\}$	3/5
0	1	1	$\{b, c\}$	2/5
1	0	0	$\{a\}$	3/5
1	0	1	$\{a, c\}$	1/5
1	1	0	$\{a, b\}$	2/5
1	1	1	$\{a, b, c\}$	1/5

Brute-Force: Example

Note that itemsets $\emptyset, \{a\}, \{b\}, \{c\}$ bring us no new knowledge.

We are only interested in itemsets with $|\mathcal{I}| \geq 2$, i.e.,

$\{b, c\}, \{a, c\}, \{a, b\}$ and $\{a, b, c\}$.

All the possible rules are:

Rule	$\text{sup}(r)$	$\text{conf}(r)$
$r_1 : b \rightarrow c$	$2/5$	$2/4 = 50\%$
$r_2 : c \rightarrow b$	$2/5$	$2/2 = 100\%$
$r_3 : a \rightarrow c$	$1/5$	$1/3 \approx 33\%$
$r_4 : c \rightarrow a$	$1/5$	$1/2 = 50\%$
$r_5 : a \rightarrow b$	$2/5$	$2/3 \approx 67\%$
$r_6 : b \rightarrow a$	$2/5$	$2/4 = 50\%$
$r_7 : a \rightarrow \{b, c\}$	$1/5$	$1/3 \approx 33\%$
$r_8 : b \rightarrow \{a, c\}$	$1/5$	$1/4 = 25\%$
$r_9 : c \rightarrow \{a, b\}$	$1/5$	$1/2 = 50\%$
$r_{10} : \{a, b\} \rightarrow c$	$1/5$	$1/2 = 50\%$
$r_{11} : \{a, c\} \rightarrow b$	$1/5$	$1/1 = 100\%$
$r_{12} : \{b, c\} \rightarrow a$	$1/5$	$1/2 = 50\%$

Let $\text{minsup} = 40\%$ and $\text{minconf} = 60\%$. The only strong rules are r_2 and r_5 .

This completes our first attempt to handle an association rule mining task using the Brute-force approach.

Such method is prohibitively slow when $|\mathcal{U}|$ is large, as the number of possible itemsets $\mathcal{I}$ is $2^{|\mathcal{U}|}$ (cf. table on p.12). For each possible $\mathcal{I}$, we will generate ≥ 2 possible rules.

Thus, we will need to find a more **efficient way** for rule mining.

Apriori Principles

As seen from the above, association rule mining can be recast as two sub-problems:

- **Frequent Itemset Mining:** Find all $\mathcal{I}$ with $\text{sup}(\mathcal{I}) \geq \text{minsup}$, and store them as frequent itemset $\mathcal{F}$.
- **Rule Generation:** For each frequent itemset in $\mathcal{F}$, generate rules with $\text{sup}(r) \geq \text{minconf}$.

We will describe the **Apriori algorithm** that attacks Frequent itemset mining. Then, we present how to handle rule generation.

Lattice Representation of Itemsets

- A lattice structure can be used to list out all possible itemsets.
- We start by a null node $\emptyset$ at the root level. Next, it produces children of all the 1-itemsets, i.e., all the single-element sets in $\mathcal{U}$.
- The next level is generated by the union of two nodes at it's mother nodes. For the k -th layer, combine nodes at the $(k - 1)$ -th layer to obtain unique nodes.

Lattice Representation

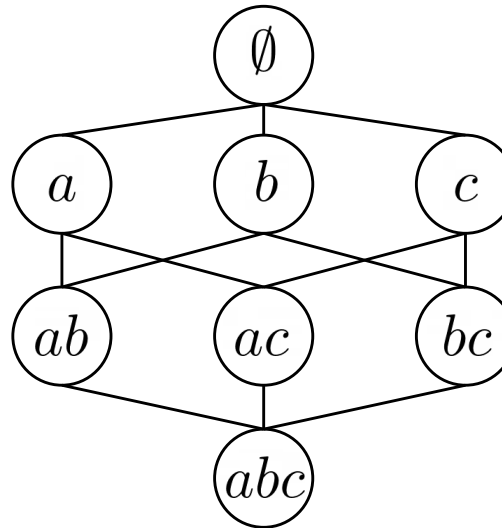
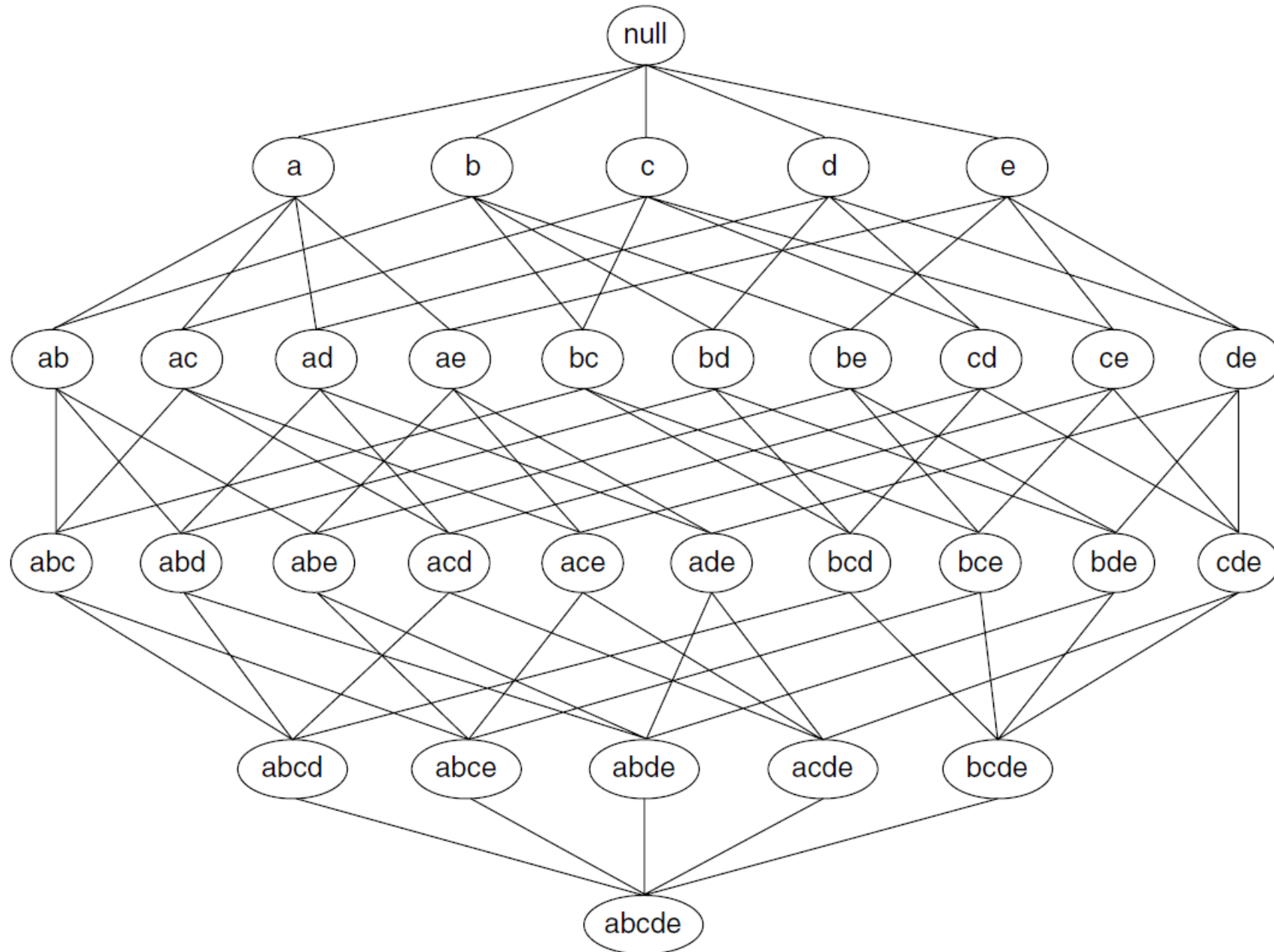


Figure: An itemset lattice with $d = 3$, where $\mathcal{U} = \{a, b, c\}$.

- A $\mathcal{T}$ with $\mathcal{U}$ has $2^{|\mathcal{U}|} - 1$ nodes in the lattice
- The brute-force approach would have search over the whole lattice before concluding which nodes are frequent, which is computationally insane.
- See the lattice of $\mathcal{U} = \{a, b, c, d, e\}$ on the next page

Lattice Representation



Main Result: Apriori Theorem

Theorem. (Apriori).

For itemsets $\mathcal{I}_1, \mathcal{I}_2 \subset \mathcal{U}$ and $\mathcal{I}_1 \subseteq \mathcal{I}_2$, it holds that

$$\text{sup}(\mathcal{I}_1) \geq \text{sup}(\mathcal{I}_2).$$

In other words:

- If $\mathcal{I}_2$ is **frequent**, then $\mathcal{I}_1$ **must be frequent**.
- If $\mathcal{I}_1$ is **not frequent**, then $\mathcal{I}_2$ **cannot be frequent**.

Intuitive examples:

- if $\{beer, egg\}$ is frequent, then both $\{beer\}$ and $\{egg\}$ must be frequent
- if $\{beer\}$ is not frequent, then $\{beer, egg\}$, $\{beer, lemon\}$, $\{beer, jam\}$ cannot be frequent

Lattice Pruning by Apriori

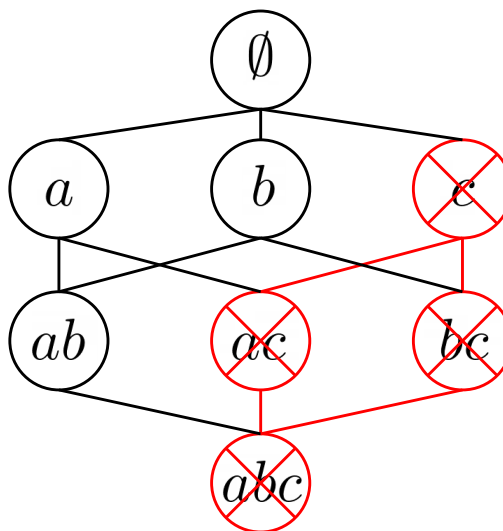


Figure: The itemset lattice of $\mathcal{U} = \{a, b, c\}$ with nodes containing $\{c\}$ eliminated.

- consider $\mathcal{U} = \{a, b, c\}$ and the itemset lattice on page 16
- suppose $\{c\}$ is **not frequent**, or, $\text{sup}(\{c\}) < \text{minsup}$
- we can then eliminate all the nodes containing $\{c\}$

Observe that we have eliminated 4 nodes by using only an insufficiency of one itemset. This allows one to generate the frequent itemsets in a far more efficient manner.

Example: Frequent Itemset Mining by Apriori

$\mathcal{T} =$	TID	items
	1	$\{beer, bread\}$
	2	$\{beer, butter\}$
	3	$\{butter, milk, potato\}$
	4	$\{beer, bread, milk, onion\}$
	5	$\{beer, bread, milk\}$
	6	$\{beer, bread, butter, milk, onion\}$

- Let $\text{minsup} = 60\%$; Unique itemsets: $\mathcal{U} = \{beer, bread, butter, milk, potato, onion\}$.
- only $\{beer\}$, $\{bread\}$, and $\{milk\}$ are frequent 1-itemsets, i.e.,

$$\mathcal{F}_1 = \{\{beer\}, \{bread\}, \{milk\}\}.$$

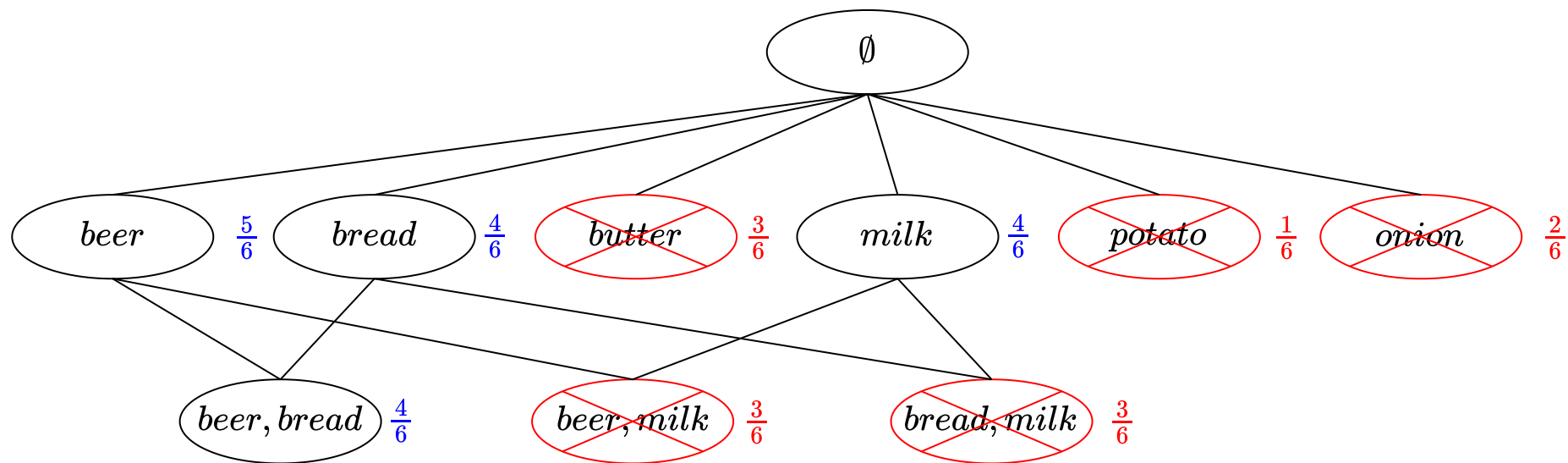
- possible 2-itemsets are:

$$\mathcal{I}_1^{(2)} = \{beer, bread\}, \quad \mathcal{I}_2^{(2)} = \{beer, milk\}, \quad \mathcal{I}_3^{(2)} = \{bread, milk\}.$$

Their corresponding supports are calculated as

$$\text{sup}(\{beer, bread\}) = 4/6, \quad \text{sup}(\{beer, milk\}) = 3/6, \quad \text{sup}(\{bread, milk\}) = 3/6.$$

We see that the only frequent 2-itemset is $\mathcal{F}_2 = \{\{beer, bread\}\}$.



- Above is the lattice corresponding to the prescribed Apriori procedures; the fractions beside the nodes are the corresponding $\text{sup}(\cdot)$ measures
- Note that the nodes we pruned in level-1 will not have any descendant node; and similarly children nodes can only be combined by two parent nodes with $\text{sup}(\cdot) \geq \text{minsup}$
- This is because of the Apriori Theorem: Let $\mathcal{I}_1 \subseteq \mathcal{I}_2$, if $\mathcal{I}_1$ is **not frequent**, then $\mathcal{I}_2$ **cannot be frequent**.

The number of lattice nodes we visited in this example is merely $6 + 3 = 9$, which is far less than $2^6 - 1 = 63$ nodes required by the brute-force approach.

Apriori Algorithm

To sum up, the Apriori algorithm is executed to search for $\mathcal{F}$ in the following order:

Step 1: Set $k = 1$.

Step 2: Find all the frequent itemsets $\mathcal{F}_k$ at the k -th level; go to step 4 if $\mathcal{F}_k = \emptyset$.

Step 3: Set $k = k + 1$; go to step 4 when $k \geq |\mathcal{U}|$, repeat step 2 otherwise.

Step 4: Return $\mathcal{F} = \mathcal{F}_1 \cup \mathcal{F}_2 \cup \dots \cup \mathcal{F}_k$.

The remaining problem: [rule generation](#).

Rule Generation

Given a frequent k -itemset $\mathcal{I} = \{\ell_1, \dots, \ell_k\}$, it produces $2^{|\mathcal{I}|} - 2$ candidate rules.

Recall that a rule shall satisfy

$$r : \mathcal{I}_1 \rightarrow \mathcal{I}_2, \quad \mathcal{I}_1 \cap \mathcal{I}_2 = \emptyset.$$

If $\mathcal{I}_1, \mathcal{I}_2 \subset \mathcal{I}$, all possible rules satisfy $\mathcal{I}_2 = \mathcal{I} \setminus \mathcal{I}_1$.

rule number	$\mathcal{I}_1$	$\rightarrow$	$\mathcal{I}_2 = \mathcal{I} \setminus \mathcal{I}_1$
1	$\{\}$	$\rightarrow$	$\{\ell_1, \ell_2, \dots, \ell_k\}$
2	$\{\ell_1\}$	$\rightarrow$	$\{\ell_2, \ell_3, \dots, \ell_k\}$
3	$\{\ell_2\}$	$\rightarrow$	$\{\ell_1, \ell_3, \dots, \ell_k\}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
k	$\{\ell_k\}$	$\rightarrow$	$\{\ell_1, \dots, \ell_{k-1}\}$
$k+1$	$\{\ell_1, \ell_2\}$	$\rightarrow$	$\{\ell_3, \ell_4, \dots, \ell_k\}$
$k+2$	$\{\ell_1, \ell_3\}$	$\rightarrow$	$\{\ell_2, \ell_4, \dots, \ell_k\}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
2^k	$\{\ell_1, \ell_2, \dots, \ell_k\}$	$\rightarrow$	$\{\}$

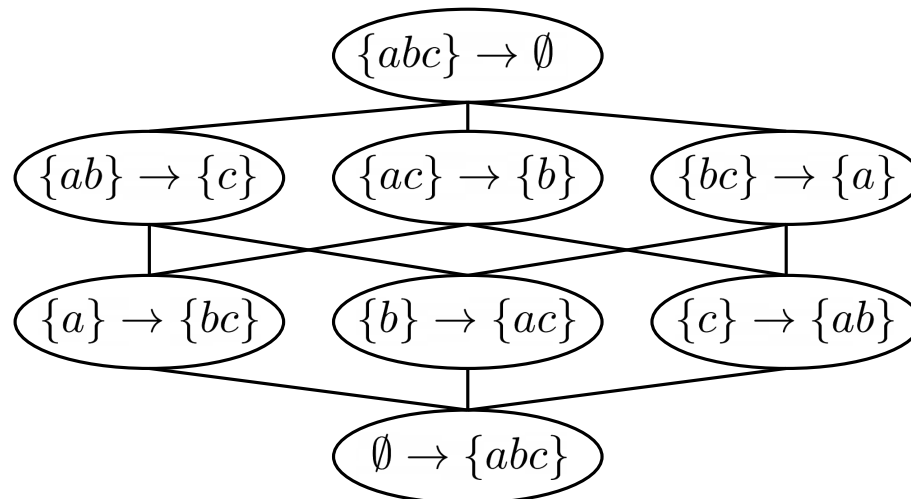
Illustration: Possible Rules from a k -itemset

Rule Generation: Lattice Representation

- the rules $\{\} \rightarrow \mathcal{I}$ and $\mathcal{I} \rightarrow \{\}$ are meaningless, hence we have $2^k - 2$ candidate rules generated by a frequent k -itemset
- these rules can be arranged by the cardinality of $\mathcal{I}_1$, and provide a lattice representation
- example: let $\mathcal{I} = \{a, b, c\}$, the generated candidate rules are:

$$\begin{array}{cccc} \{a, b, c\} \rightarrow \emptyset & \{a\} \rightarrow \{b, c\} & \{b\} \rightarrow \{a, c\} & \{c\} \rightarrow \{a, b\} \\ \{a, b\} \rightarrow \{c\} & \{a, c\} \rightarrow \{b\} & \{b, c\} \rightarrow \{a\} & \emptyset \rightarrow \{a, b, c\} \end{array}$$

and the lattice representation is:



Apriori for Rule Generation

Theorem. (Apriori for confidence).

Given a frequent itemset $\mathcal{I}$, suppose a rule

$$\mathcal{I}_1 \rightarrow \mathcal{I}_2 = \mathcal{I} \setminus \mathcal{I}_1$$

has a confidence lower than threshold minconf. Then,

$$\text{conf}(\mathcal{I}'_1 \rightarrow \mathcal{I}'_2) \leq \text{conf}(\mathcal{I}_1 \rightarrow \mathcal{I}_2) < \text{minconf} \quad \forall \mathcal{I}'_1 \subseteq \mathcal{I}_1, \mathcal{I}'_2 = \mathcal{I} \setminus \mathcal{I}'_1.$$

In other words, if $\mathcal{I}_1 \rightarrow \mathcal{I}_2$ is not strong, then, for any subset of $\mathcal{I}_1$, the rule $\mathcal{I}'_1 \rightarrow \mathcal{I}_2$ cannot be strong.

Proof: use the definition of confidence:

$$\text{conf}(\mathcal{I}_1 \rightarrow \mathcal{I}_2) = \frac{\text{sup}(\mathcal{I})}{\text{sup}(\mathcal{I}_1)} \geq \frac{\text{sup}(\mathcal{I})}{\text{sup}(\mathcal{I}'_1)} = \text{conf}(\mathcal{I}'_1 \rightarrow \mathcal{I}'_2).$$

The middle $\geq$ is due to $\text{sup}(\mathcal{I}_1) \leq \text{sup}(\mathcal{I}'_1)$ for all $\mathcal{I}'_1 \subseteq \mathcal{I}_1$. This completes the proof.

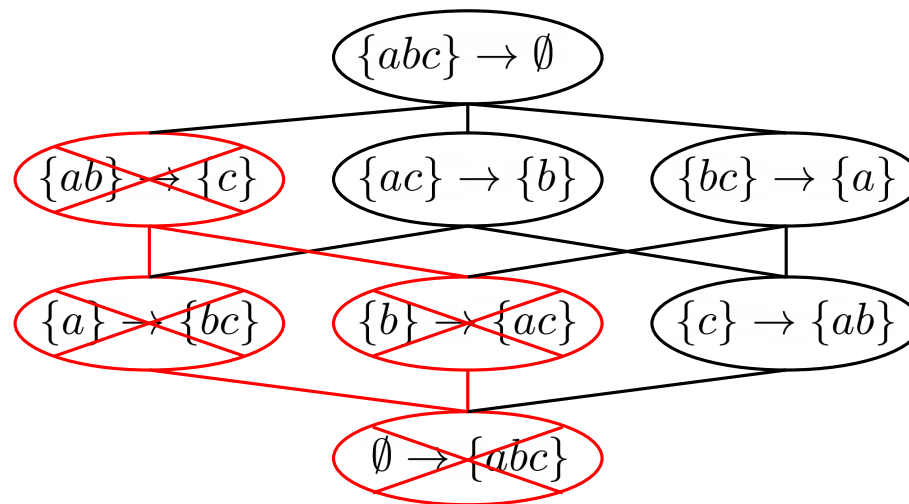
Example

- Suppose we know $\text{conf}(\{a, b\} \rightarrow \{c\}) < \text{minconf}$, then –

$$\text{conf}(\{a\} \rightarrow \{b, c\}) < \text{minconf},$$

$$\text{conf}(\{b\} \rightarrow \{a, c\}) < \text{minconf}.$$

This is illustrated by the lattice pruning:



- Again, the Apriori method allows us to prune most rules from the whole lattice.

Example: Rule Generation by Apriori

$\mathcal{T} =$	TID	items
	1	$\{beer, bread\}$
	2	$\{beer, butter\}$
	3	$\{butter, milk, potato\}$
	4	$\{beer, bread, milk, onion\}$
	5	$\{beer, bread, milk\}$
	6	$\{beer, bread, butter, milk, onion\}$

Let $\text{minsup} = 50\%$, $\text{minconf} = 90\%$; the only frequent 3-itemset is $\mathcal{I} = \{beer, bread, milk\}$, with $\text{sup}(\mathcal{I}) = 3/6$. Consider the (first layer) rules:

$$r_1 : \{beer, bread\} \rightarrow \{milk\}, \quad r_2 : \{beer, milk\} \rightarrow \{bread\}, \quad r_3 : \{bread, milk\} \rightarrow \{beer\}.$$

We can compute their corresponding confidence as:

$$\text{conf}(r_1) = 75\%, \quad \text{conf}(r_2) = \text{conf}(r_3) = 100\%.$$

This means that the descendant nodes of $\{beer, bread\} \rightarrow \{milk\}$ are pruned. In the next layer, we only consider the rule $\{milk\} \rightarrow \{beer, bread\}$. The confidence is $\text{conf}(\{milk\} \rightarrow \{beer, bread\}) = (3/6)/(4/6) = 75\% < \text{minconf}$. Thus, we conclude the only strong rules are r_2 and r_3 .